

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1104 Fundamentos de Sistemas Computacionales

I Semestre 2026

**Documento de Atributos de Acreditación:
Conocimiento de Ingeniería (CI)**

Proyecto 2 - Radar 2D

Profesor: Leonardo Araya

Integrantes:

- Sebastián Hernández Ballesteros
- Fariathna Venegas Venegas
 - Gabriel Ramirez Soto

2. Introducción

Este documento tiene la finalidad de documentar los atributos del Proyecto #2 de Fundamentos de sistemas computacionales y satisfacer los "Conocimiento de Ingeniería (CI)" en el desarrollo de un radar 2D utilizando una plataforma de hardware (Arduino) y una interfaz gráfica (GUI) en Python. El proyecto requirió la integración de conceptos de variables físicas, matemáticas y ciencias de la computación para resolver problemas complejos, tales como la transformación de las coordenadas en tiempo real, el cálculo de trayectorias y el manejo de concurrencia en la comunicación de datos. A continuación, se identifican las principales barreras técnicas encontradas durante la fase de software y hardware, las alternativas planteadas para cada una y la justificación técnica detrás de las soluciones implementadas.

3. Identificación de problemas

- **Problema 1. Dependencia de hardware en desarrollo remoto y bloqueos por espera de señal**

Debido a que el equipo trabajó de manera remota y el hardware físico (Arduino y sensores) no estaba disponible para la programación de la interfaz, por cuestión de tiempo por ser final de semestre fue necesario crear un entorno de simulación. El problema técnico radicaba en que el sistema, al intentar leer el puerto serial sin el Arduino conectado, se quedaba bloqueado esperando una señal que nunca llegaría. Se requería una arquitectura que permitiera alternar entre "datos falsos" (simulados) y "datos reales" de forma dinámica, sin que el programa colapsara o se congelara por la falta de conexión física.

- **Problema 2. Implementación de ciclos de actualización asíncronos en la interfaz gráfica**

Para que el radar dibujara el barrido y los objetos de forma fluida, se necesitaba actualizar la pantalla continuamente. El problema fue que utilizar ciclos tradicionales de programación (como un while True con time.sleep()) congelaba por completo el ciclo de eventos principal de Tkinter (mainloop), provocando que la ventana dejara de responder y no se mostrarán los gráficos ni los datos de la ventana principal.

- **Problema 3. Inestabilidad en las lecturas del sensor ultrasónico.**

A la hora de que el sensor ultrasónico HC-SR04 leía los datos, cuando reportaba los datos de la distancia podía ser un poco inconsistente, aunque estuviera quieto. Es decir, que subiera o bajara entre 1 a 2 unidades. Aunque es un cambio muy pequeño entre las mediciones, si afecta la precisión del radar. El problema se agravaba cuando se ponía a funcionar el servo. La principal causa de la inestabilidad en las mediciones del sensor ultrasónico HC-SR04 es que el sensor funciona enviando ondas de sonido y midiendo el tiempo que tardan en regresar después de rebotar en un objeto. El

problema es que esta onda se puede ver afectada por factores como: Ecos, interferencias y reflexiones.

4. Planteamiento de la solución

Opciones para el Problema 1 (Dependencia de hardware y bloqueos)

- **Opción A:** Crear dos archivos de código fuente en Python totalmente separados: uno dedicado exclusivamente a la simulación con lecturas inventadas y otro para la conexión real con el Arduino, ejecutando el que correspondiera según la necesidad.
- **Opción B:** Escribir los valores simulados directamente dentro de las funciones de lectura principales y borrarlos o comentarlos manualmente cada vez que se necesitara conectar el Arduino.
- **Opción C:** Implementar una bandera global (`SIMULATION_MODE = True/False`) y utilizar un hilo de ejecución secundario. Este hilo actúa como un juez que, dependiendo del estado de la bandera, ejecuta la función `_simular()` o `_leer_arduino()`, depositando los datos (reales o inventados) en una cola (`queue.Queue`) independiente del proceso principal.

Opciones para el Problema 2 (Ciclos de actualización asíncronos)

- **Opción A:** Utilizar un ciclo `while True` principal y forzar la actualización gráfica manual llamando a la función del código `ventana.update()` en cada pasada del ciclo para evitar que Tkinter se congele.
- **Opción B:** Importar una librería externa especializada en el desarrollo de videojuegos (como `pygame`), la cual posee un ciclo de renderizado nativo (ya integrado en la librería) diseñado para manejar gráficos continuos sin los bloqueos típicos de una GUI.
- **Opción C:** Aprovechar el método nativo `after( milisegundos, función )` de Tkinter, esto permite crear métodos recursivos (como `_loop_principal` y `_loop_animacion_espera`) que se programan a sí mismos para ejecutarse cada 50 milisegundos, cediendo el control inmediatamente al mainloop tras procesar los datos de la cola y dibujar en el Canvas.

Opciones para el Problema 3 (Inestabilidad en las lecturas del sensor ultrasónico)

- **Opción A:** Cambiar el servomotor por uno de mejor calidad, que realice las angulaciones más precisas y que no genere tanta vibración. Un servomotor de mayor calidad puede posicionarse con mayor exactitud y estabilidad, reduciendo significativamente los errores introducidos durante el proceso de medición.
- **Opción B:** Antes de comenzar las lecturas colocar un `delay`, este tiempo de espera permite que el servomotor alcance la posición deseada y reduzca sus vibraciones antes de que el sensor ultrasónico realice la medición.

- **Opción C:** Bajar la velocidad con la que el servo gira, se reducen las aceleraciones y desaceleraciones bruscas, disminuyendo las vibraciones mecánicas que se transmiten al sensor ultrasónico.

5. Selección y justificación de la mejor opción

Solución seleccionada para el Problema 1.

Se seleccionó la opción C (Uso de bandera global y cola de subprocesos). La Opción A se descartó porque mantener dos archivos de código separados genera duplicidad, cualquier mejora visual en el radar obligaría a programarla dos veces, aumentando el margen de error. La Opción B es una mala práctica de ingeniería, ya que modificar el código fuente constantemente para pruebas es ineficiente y propenso a causar errores de sintaxis antes de la presentación final. La Opción C resolvió el problema de forma más práctica, al utilizar hilos y la estructura queue.Queue, el proceso que dibuja la interfaz nunca se entera de dónde provienen los datos. Esto permitió probar el software al 100% en remoto mediante funciones simuladas y el día de la integración, bastó con cambiar `SIMULATION_MODE = False`, para que el sistema leyera el Arduino de inmediato, sin que el programa principal sufriera bloqueos por esperas de conexión en el puerto serial.

Solución seleccionada para el Problema 2.

Se seleccionó la Opción C (Uso del método `after` de Tkinter). La Opción A (forzar el `update()` en un bucle) no es recomendada porque satura el procesador, causando comportamientos erráticos al minimizar la ventana o al interactuar con otros elementos. La Opción B se rechazó porque introducir `pygame` añadía una dependencia muy pesada e innecesaria para un proyecto que debía centrarse en una interfaz estándar, además de la inexperiencia que suponía utilizar esta librería que nunca hemos explorado como grupo optamos por utilizar las herramientas brindadas tanto en este curso como en el de Introducción a la Programación, la Opción C demostró ser la más estable. Al utilizar `self.root.after()`, las funciones se encolan ordenadamente en la fila de tareas de Tkinter. Esto garantizó que la lectura de la cola de datos y el redibujado de la pantalla ocurran exactamente de forma asíncrona cada 50 ms, manteniendo una experiencia lo más fluida posible, limitantes de refresco de frames por segundo (aproximadamente 45 fps), sin sacrificar rendimiento del equipo.

Solución seleccionada para el Problema 3

Se seleccionó la opción B (Utilizar un delay antes de comenzar la lectura). La opción A implicaba hacer un gasto adicional de dinero, además de que ya todo el hardware estaba conectado / ensamblado cuando se detectó el problema, por ello se descartó. La opción C era una buena idea, sin embargo para erradicar el problema había que bajarle considerablemente la velocidad y hacía que el tiempo para concretar el barrido subiera a aproximadamente 26 segundos, lo que era muy lento. Finalmente se concluyó que la opción B era la mejor, ya que La solución implementada mediante un retardo de 40 ms elimina gran parte de las

vibraciones sin requerir componentes adicionales ni cambios significativos en el diseño del sistema.